

1. A* IMPLEMENTATION: MISPLACED TILES

```
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0) # 0 = blank

def h(state):
    # number of misplaced tiles (blank not counted)
    return sum(1 for i, t in enumerate(state) if t != 0 and t != GOAL[i])

def neighbors(state):
    b = state.index(0)
    r, c = divmod(b, 3)
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            s = list(state)
            n = nr * 3 + nc
            s[b], s[n] = s[n], s[b]
            yield tuple(s)

def solvable(state):
    t = [x for x in state if x != 0]
    inv = sum(t[i] > t[j] for i in range(len(t)) for j in range(i + 1, len(t)))
    return inv % 2 == 0

def astar(start):
    if not solvable(start):
        return None, 0
    heap = [(h(start), 0, start)] # (f, g, state)
    parent = {start: None}
    best_g = {start: 0}
```

```

closed = set()
expanded = 0
while heap:
    f, g, state = heapq.heappop(heap)
    if state in closed:
        continue
    closed.add(state)
    expanded += 1
    if state == GOAL:
        path = []
        while state:
            path.append(state)
            state = parent[state]
        return path[::-1], expanded
    for nxt in neighbors(state):
        ng = g + 1          # g(n) = depth
        if nxt not in best_g or ng < best_g[nxt]:
            best_g[nxt] = ng
            parent[nxt] = state
            heapq.heappush(heap, (ng + h(nxt), ng, nxt)) # f = g + h
return None, expanded

```

```

def show(state):
    for i in range(0, 9, 3):
        print(*[x if x else "_" for x in state[i:i + 3]])
    print()

```

```

start = (1, 2, 3,
        4, 0, 6,
        7, 5, 8)

```

```

path, expanded = astar(start)

if path is None:
    print("Unsolvable")
else:
    print("Heuristic: Misplaced Tiles")
    print("Moves:", len(path) - 1, "| Nodes expanded:", expanded, "\n")
    for step, s in enumerate(path):
        print(f"Step {step}: g={step}, h={h(s)}, f={step + h(s)}")
        show(s)

```

OUTPUT :

```

Heuristic: Misplaced Tiles
Moves: 2 | Nodes expanded: 3

Step 0: g=0, h=2, f=2
1 2 3
4 _ 6
7 5 8

Step 1: g=1, h=1, f=2
1 2 3
4 5 6
7 _ 8

Step 2: g=2, h=0, f=2
1 2 3
4 5 6
7 8 _

```

2. A* IMPLEMENTATION: MANHATTAN DISTANCE

```
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 0) # 0 = blank

GOAL_POS = {t: divmod(i, 3) for i, t in enumerate(GOAL)}

def h(state):
    # sum of Manhattan distances of every tile from its goal position
    total = 0
    for i, t in enumerate(state):
        if t != 0:
            r, c = divmod(i, 3)
            gr, gc = GOAL_POS[t]
            total += abs(r - gr) + abs(c - gc)
    return total

def neighbors(state):
    b = state.index(0)
    r, c = divmod(b, 3)
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            s = list(state)
            n = nr * 3 + nc
            s[b], s[n] = s[n], s[b]
            yield tuple(s)

def solvable(state):
    t = [x for x in state if x != 0]
    inv = sum(t[i] > t[j] for i in range(len(t)) for j in range(i + 1, len(t)))
    return inv % 2 == 0
```

```

def astar(start):
    if not solvable(start):
        return None, 0
    heap = [(h(start), 0, start)]    # (f, g, state)
    parent = {start: None}
    best_g = {start: 0}
    closed = set()
    expanded = 0
    while heap:
        f, g, state = heapq.heappop(heap)
        if state in closed:
            continue
        closed.add(state)
        expanded += 1
        if state == GOAL:
            path = []
            while state:
                path.append(state)
                state = parent[state]
            return path[::-1], expanded
        for nxt in neighbors(state):
            ng = g + 1            # g(n) = depth
            if nxt not in best_g or ng < best_g[nxt]:
                best_g[nxt] = ng
                parent[nxt] = state
                heapq.heappush(heap, (ng + h(nxt), ng, nxt))    # f = g + h
    return None, expanded

def show(state):
    for i in range(0, 9, 3):

```

```

        print(*[x if x else "_" for x in state[i:i + 3]])
    print()

start = (1, 2, 3,
        4, 0, 6,
        7, 5, 8)

path, expanded = astar(start)

if path is None:
    print("Unsolvable")
else:
    print("Heuristic: Manhattan Distance")
    print("Moves:", len(path) - 1, "| Nodes expanded:", expanded, "\n")
    for step, s in enumerate(path):
        print(f"Step {step}: g={step}, h={h(s)}, f={step + h(s)}")
        show(s)

```

OUTPUT :

```

Heuristic: Manhattan Distance
Moves: 2 | Nodes expanded: 3

Step 0: g=0, h=2, f=2
1 2 3
4 _ 6
7 5 8

Step 1: g=1, h=1, f=2
1 2 3
4 5 6
7 _ 8

Step 2: g=2, h=0, f=2
1 2 3
4 5 6
7 8 _

```

